

# Stepping Desk — AI Email Summarization Platform

## Engineering Status & Code Flow — 2026-07-16

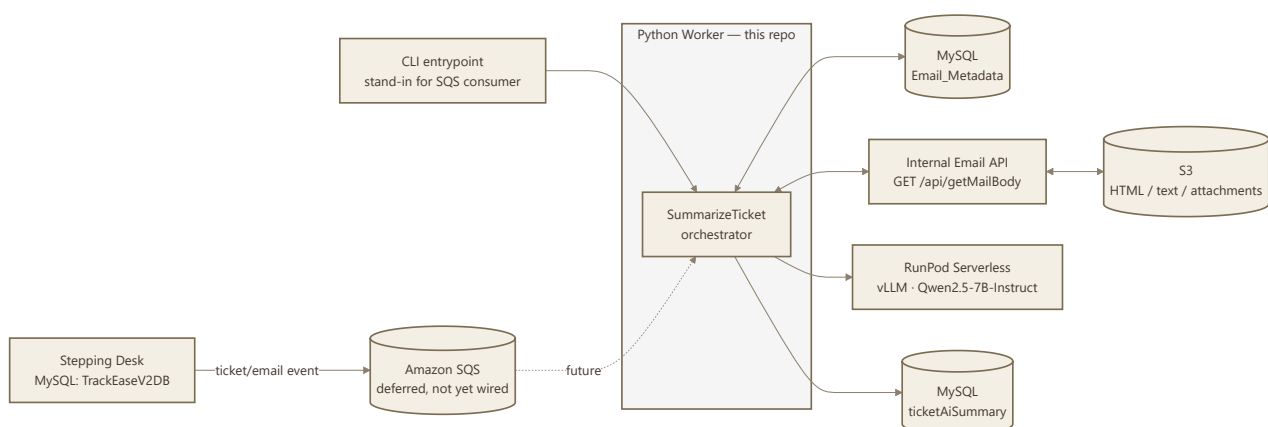
Source of truth for narrative decisions remains `CLAUDE.md`. This document is a point-in-time snapshot for onboarding / stakeholder review, verified against the actual repo state (test run, mypy, ruff, git log) at time of writing — not just copied from docs.

### 1. Where we are, in one paragraph

Phase 1 (AI-generated ticket summaries, no classification/RAG yet) is **implemented, unit-tested, and has been run successfully end-to-end against real staging infrastructure** — real MySQL, the real internal Email API, and the real RunPod/vLLM endpoint — producing a correct summary row for a live ticket. The pipeline is driven today by a CLI entrypoint rather than SQS (SQS wiring is deferred to the team). Six real bugs were found and fixed via that staging run that unit tests structurally could not have caught (wrong env loading, missing DB column, wrong model-name casing, wrong context length, a vLLM decoding quirk, a logging gap). The Email API contract has changed **twice** since integration began (2026-07-15, 2026-07-16); the codebase has been updated for both, the second update is **staged but uncommitted**, and has been validated against real staging data for the email-fetch path specifically (ticket 239908) — the one remaining unknown on that front is the not-yet-available/RYW signal shape, which hasn't been observed for real yet.

**Verified right now:** 184/184 unit tests passing, `mypy --strict` clean, `ruff` clean, across 34 source files / ~2,500 lines in `src/summarizer`. 8 integration tests for the MySQL repository are written but still unexecuted (no local Docker).

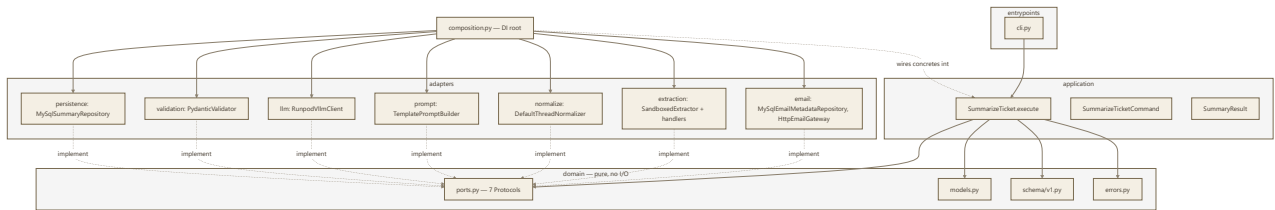
### 2. System context



- **Trigger** (future): one SQS message per new/replied email — `{ticketId, emailMetaId, messageId}`.
- **Today:** `python -m summarizer.entrypoints.cli --ticket-id N --email-meta-id N --message-id ID` drives the identical orchestrator logic the SQS consumer will call later.
- **Volume:** <100 tickets/day now, 1–20 emails/ticket, ~40k tickets to backfill later. Latency budget ~1 minute — no real-time requirement.

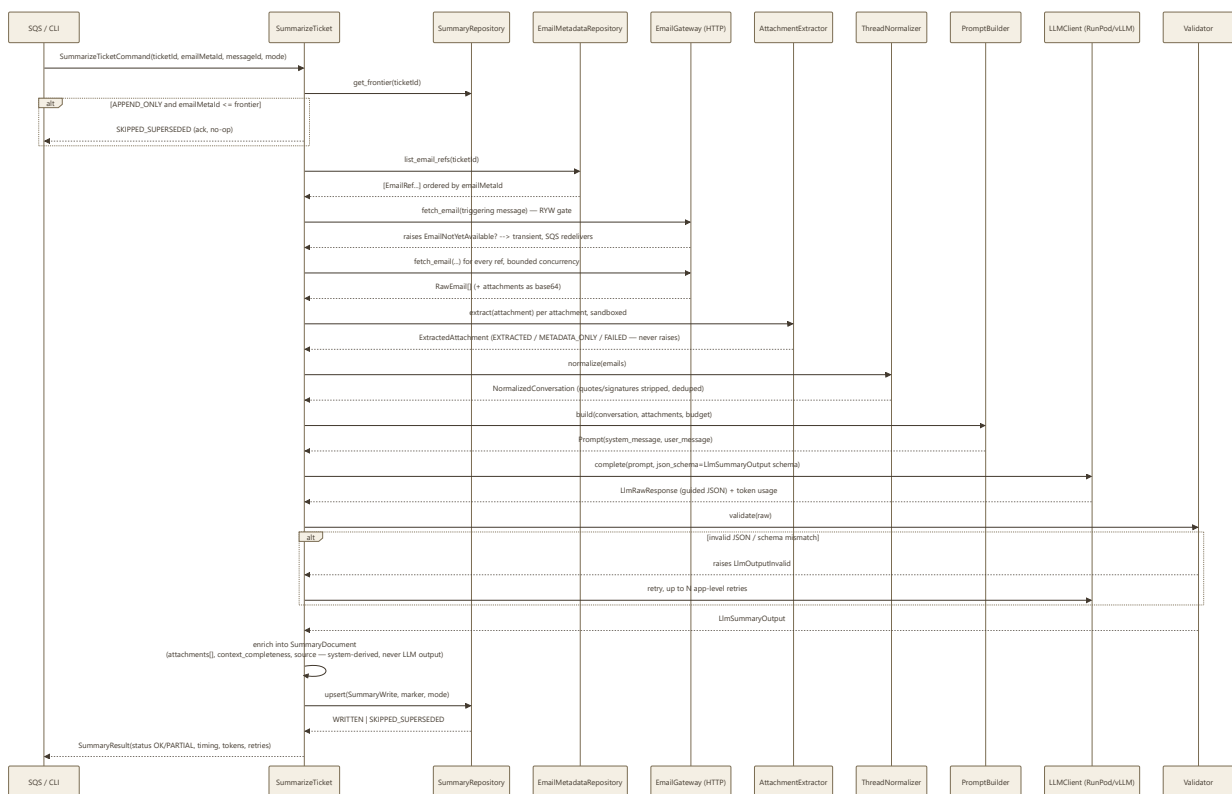
### 3. Architecture — hexagonal / clean

Dependency rule: domain depends on nothing; application depends only on domain; adapters / entrypoints depend inward. composition.py is the *only* place concrete adapters are named.

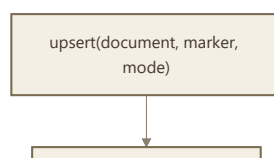


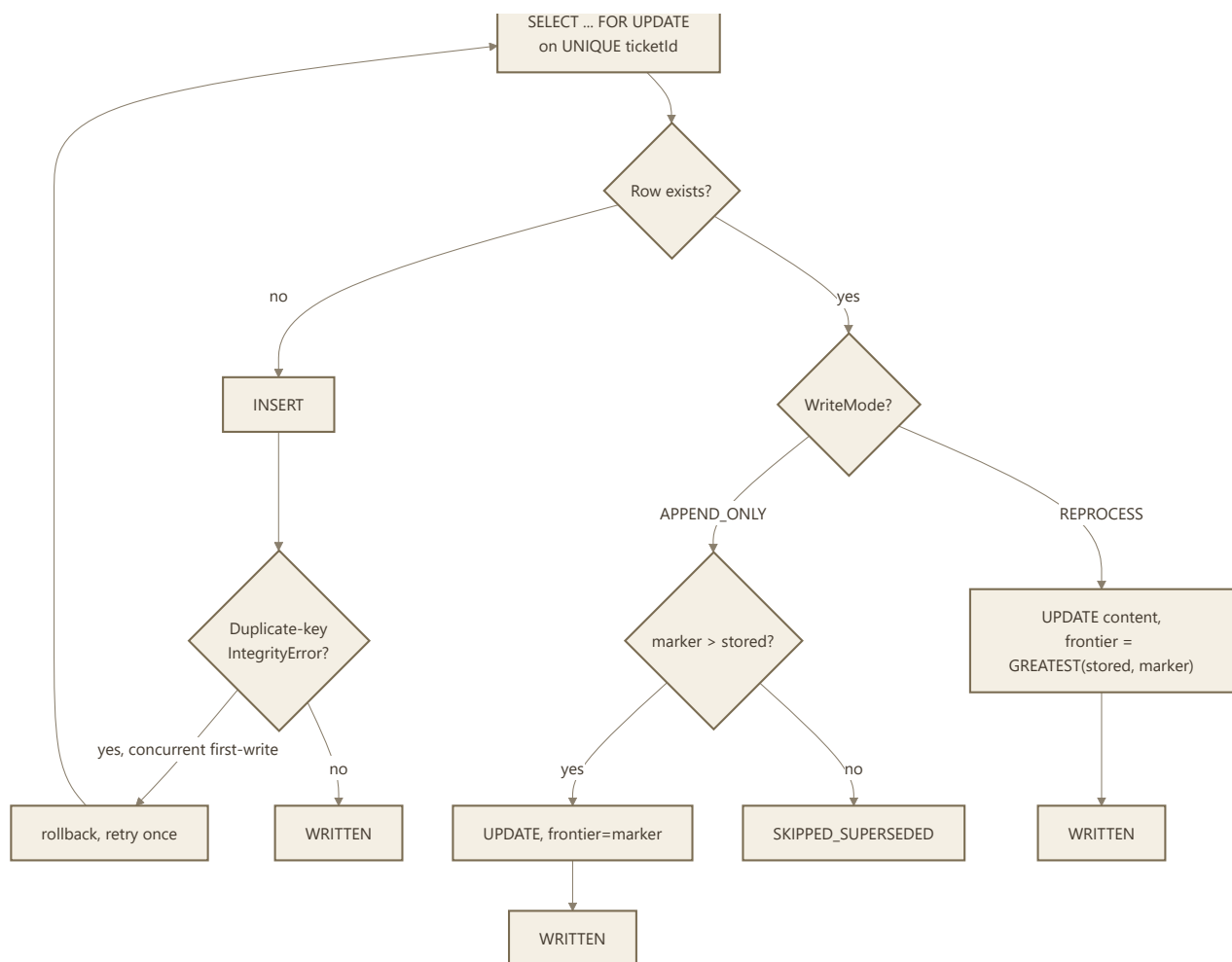
**Why this shape:** the orchestrator (SummarizeTicket) never imports requests, pymysql, or any SDK directly — it only knows the seven Protocol interfaces in domain/ports.py. That's what makes the whole pipeline testable with fakes (184 unit tests, zero real I/O) and swappable (e.g. RunPod → a different LLM host is a one-adapter change, not a rewrite).

### 4. End-to-end code flow (happy path)

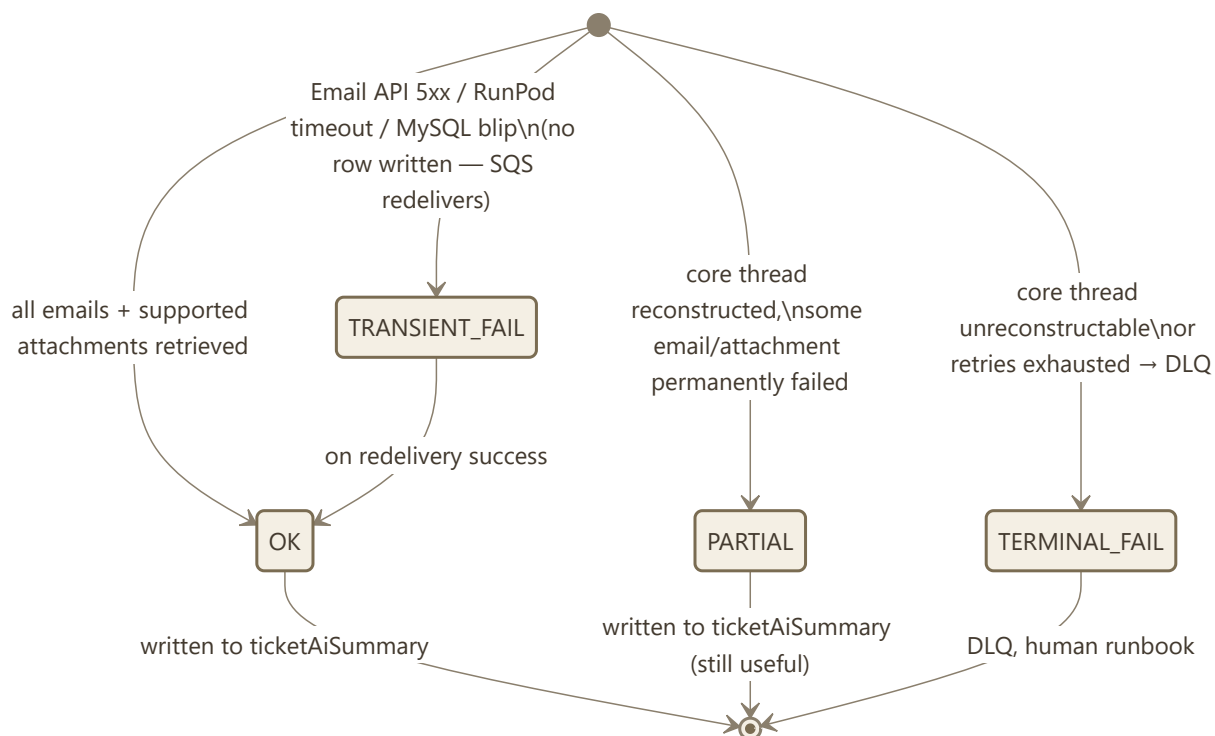


**Key correctness mechanism — compare-and-set (CAS):** emailMetaId doubles as the FK to the triggering email *and* the monotonic frontier marker. APPEND\_ONLY mode (default, used by live traffic and DLQ redrives) writes only if incoming > stored; REPROCESS mode (admin-only: prompt/model upgrades) force-overwrites content but the frontier can only move forward (GREATEST(stored, incoming)). This is what makes standard (non-FIFO) SQS safe, makes duplicate delivery merely wasteful rather than incorrect, and makes the 40k-ticket backfill idempotent/resumable for free.





## 5. Status lifecycle



Only OK and PARTIAL are ever persisted (PersistedSummaryStatus enum enforces this at the type level — the repository literally cannot be asked to write a transient/terminal row). Per-attachment failures never fail the whole ticket; they degrade in place to METADATA\_ONLY / FAILED inside context completeness.

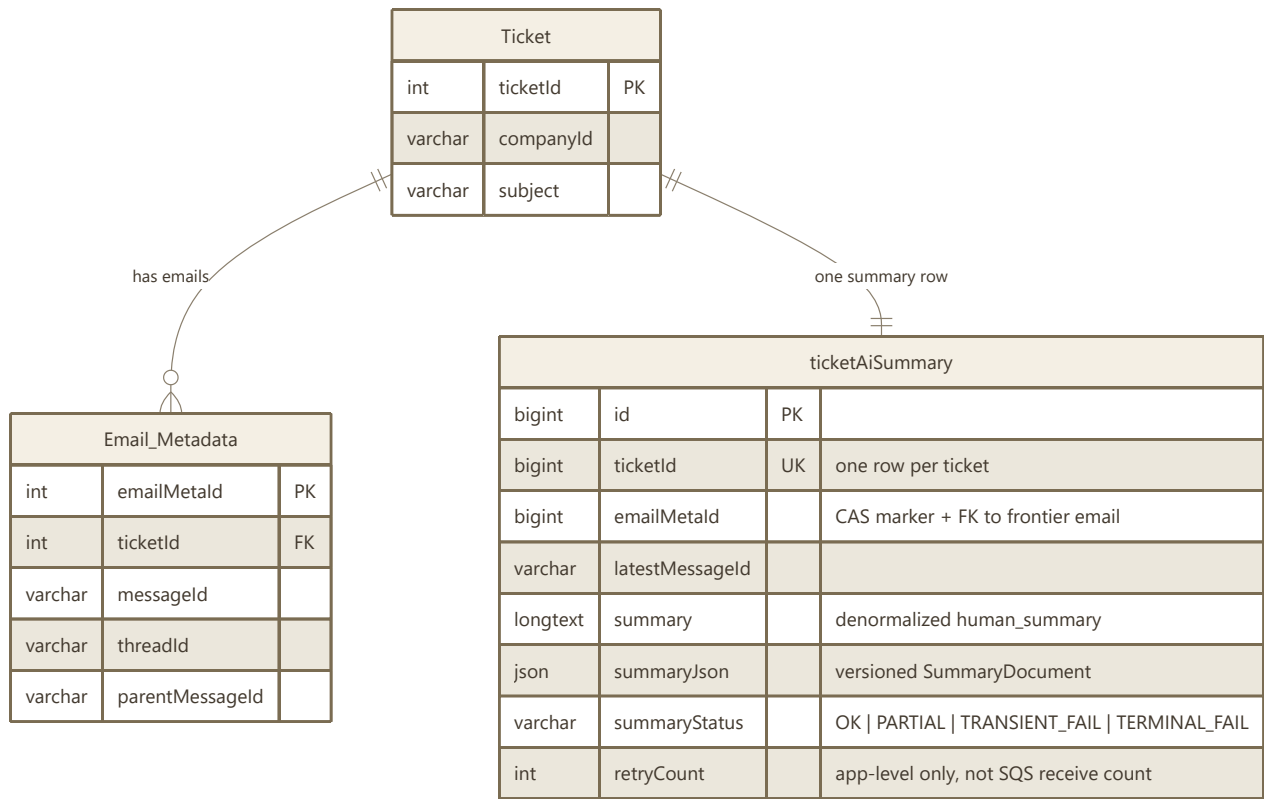
## 6. Module inventory

| Layer                | File                        | Purpose                                                                                                                                                   | Status                                                                             |
|----------------------|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| domain               | schema/v1.py                | LlmSummaryOutput (LLM-produced) + SummaryDocument (full persisted envelope). Doubles as the vLLM guided-JSON schema source.                               | ✅ stab                                                                             |
| domain               | models.py                   | RawEmail, RawAttachment, ExtractedAttachment, NormalizedConversation, Prompt (system/user split), EmailRef                                                | ✅ stab                                                                             |
| domain               | ports.py                    | 7 Protocols:<br>EmailMetadataRepository, EmailGateway, AttachmentExtractor, ThreadNormalizer, PromptBuilder, LLMClient, Validator, SummaryRepository      | ⚠️ uncc<br>docstrin<br>for the ;<br>contract                                       |
| domain               | errors.py                   | Two-branch hierarchy:<br>TransientError (redeliver) vs TerminalError (DLQ), incl. SummaryPersistenceTransient for wrapped MySQL codes 1205/1213/2006/2013 | ✅ stab                                                                             |
| adapters/persistence | mysql_summary_repository.py | CAS/reprocess write logic; decide_write() factored out as a pure, DB-free function for unit testing the R1 staleness guard                                | ✅ unit-<br>integrati<br>(no loca                                                   |
| adapters/email       | mysql_email_metadata.py     | Enumerates a ticket's Email_Metadata rows ordered by emailMetaId                                                                                          | ✅ stab                                                                             |
| adapters/email       | http_email_gateway.py       | Fetches full email + attachments from the internal Email API                                                                                              | ⚠️ uncc<br>for the 2<br>getMail<br>validate<br>data (tic<br>fetch pa<br>still uncc |
| adapters/extraction  | extractor.py + handlers.py  | Sandboxed, resource-capped extraction:<br>PDF/DOCX/TXT/CSV/XLSX + images                                                                                  | ✅ stab<br>ocr on '<br>deploy -                                                     |

|                     |                                             |                                                                                                                                                                                                        |                                |
|---------------------|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|
|                     |                                             | (OCR via pytesseract). Never raises — degrades to METADATA_ONLY / FAILED                                                                                                                               | in current                     |
| adapters/normalize  | normalizer.py                               | Quote-stripping (email_reply_parser + regex fallback), signature/disclaimer removal, dedup                                                                                                             | ✅ stab                         |
| adapters/prompt     | prompt_builder.py + templates/v1/system.txt | Versioned, token-budgeted prompt for Qwen's chat template (attachments truncated before oldest history)                                                                                                | ✅ stab                         |
| adapters/llm        | runpod_vllm_client.py                       | RunpodVllmClient — synchronous POST to the OpenAI-compatible /openai/v1/chat/completions route, guided JSON via vLLM's guided_json field                                                               | ✅ conf deployn                 |
| adapters/validation | pydantic_validator.py                       | Strips markdown fences, json.loads + LlmSummaryOutput.model_validate; failure → LlmOutputInvalid (drives retry)                                                                                        | ✅ stab                         |
| application         | summarize_ticket.py, command.py, result.py  | SummarizeTicket.execute() — the orchestrator; exact flow in §4                                                                                                                                         | ✅ stab                         |
| config              | settings.py, logging_config.py              | pydantic-settings ( .env -driven, per-sub-settings load_dotenv() ), structured JSON logging                                                                                                            | ⚠ uncc EmailAp updated .../api |
| entrypoints         | cli.py                                      | Stand-in for the SQS consumer: --ticket-id --email-meta-id --message-id [--reprocess] [--triggered-by]. Maps TransientError / TerminalError → exit 1 (logged); real bugs propagate with full traceback | ✅ stab                         |
| —                   | composition.py                              | DI root — every concrete adapter named exactly once                                                                                                                                                    | ✅ stab                         |

**Not yet built:** SQS consumer entrypoint itself (deferred — team will wire up the queue later; CLI is the interim driver), transactional outbox / SummaryUpserted event (deliberately deferred — replay-from-table is the documented bootstrap path for the future vector index), classification/RAG/embeddings (explicitly out of Phase 1 scope).

## 7. Data model



Email\_Metadata and Ticket are pre-existing Stepping Desk tables (not modified structurally). ticketAiSummary is the new Phase 1 table — confirmed live and matching this shape (the summaryJson column had to be added manually via ALTER TABLE on 2026-07-13; see §9).

SummaryDocument (the JSON payload) splits strictly by provenance:

- **content:** LlmSummaryOutput — everything the LLM produced ( human\_summary , customer\_issue , current\_status , best-effort fields like timeline / pending\_actions / keywords , and a reserved-but-unpopulated classification block).
- **attachments , context\_completeness , source** — filled in by the orchestrator *after* validation, never by the model. This keeps the LLM from ever being asked to assert something it didn't actually see (e.g. "which attachments parsed").

## 8. External integration history (the volatile part)

Both external contracts changed mid-implementation; both changes are absorbed behind the adapter layer with zero blast radius to domain / application .

**Email API** — three generations so far:

1. Original: lookup by messageId alone, bare JSON array response.
2. **2026-07-15:** found to occasionally return an empty array for real emails (ticket 239907). Fixed by disambiguating with the full identifier set ( companyId , ticketId , emailMetaId , messageId , threadId ).
3. **2026-07-16 (current, uncommitted):** endpoint moved to GET /api/getMailBody ; messageId dropped from query params entirely (redundant once emailMetaId , the row's own PK, is present); response wrapped in {"status", "result"} instead of a bare array; several field renames (html → mailBody , attachments → attachment , item fields

`filename/mimetype/attachmentId/content → fileName/fileType/emailAttachmentID/fileBase64`); `fileBase64` is now a `data: URI` whose prefix must be explicitly stripped or the attachment **silently** corrupts (invalid-looking bytes decode without raising, since the `data:...;base64,` prefix characters are themselves valid base64 alphabet). Validated against a real payload (ticket 239908) and, separately, against live staging for the fetch path across 5 real emails with zero adapter errors. **Open:** whether the not-yet-available/RYW signal is still HTTP 404, a body-level `status` field on HTTP 200, or both — `_unwrap_envelope` checks both defensively, unconfirmed against a real case.

**RunPod / vLLM** — one correction:

- Initial design assumed RunPod's native `async /run + poll /status` job pattern (modeled on an unrelated prior project's `runpod_context.py`, which still sits at the repo root as a reference-only artifact, not part of the package).
- **Confirmed 2026-07-13** against the real deployment: it's actually the OpenAI-compatible route `/openai/v1/chat/completions` — a single synchronous POST, no polling. This changed `domain/models.py::Prompt` from one pre-templated string to `system_message + user_message` fields (the OpenAI route applies Qwen's chat template server-side; hand-building `<|im_start|>` markers would have double-templated).
- `max_context_tokens` corrected from an assumed 16384 to the endpoint's real 32768 (from `/v1/models`).
- `model_name` must match `/v1/models` exactly, including case (`qwen/qwen2.5-7b-instruct`, lowercase) — a casing mismatch produced a fast, misleading generic 500 rather than a clear "model not found."

## 9. Real-infra validation log

**2026-07-13 — first full green run**, ticket 239904 (3 emails, no attachments): real MySQL + real Email API + real RunPod, wrote a correct `OK` row (1707 tokens in / 261 out, ~10.2s); a second run against the same trigger correctly short-circuited to `SKIPPED_SUPERSEDED` via the real CAS frontier. Surfaced and fixed 6 bugs invisible to unit tests (all now regression-covered):

| # | Bug                                                                                                                                                                                                                                                        | Why unit tests missed it                                                                                                    |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| 1 | Nested settings ( <code>DatabaseSettings</code> , <code>RunpodSettings</code> , ...) never actually read <code>.env</code> — each sub-settings class instantiates independently and <code>pydantic-settings'</code> <code>.env</code> loading is per-class | Every unit test set OS env vars directly via <code>monkeypatch.setenv</code> , masking the <code>.env</code> -specific path |
| 2 | Live <code>ticketAiSummary</code> table was missing the <code>summaryJson</code> column                                                                                                                                                                    | No integration test had run against real schema yet                                                                         |
| 3 | <code>Settings.llm.model_name</code> casing ( <code>Qwen/Qwen2.5-7B-Instruct</code> vs live <code>/v1/models</code> 's lowercase)                                                                                                                          | Mocked LLM client in tests never validated against a real model registry                                                    |
| 4 | <code>max_context_tokens</code> was 16384 (assumed), real endpoint is 32768                                                                                                                                                                                | Never confirmed against real <code>/v1/models</code> before                                                                 |
| 5 | vLLM's <code>outlines</code> guided-decoding backend returned                                                                                                                                                                                              | Fake LLM responses in tests always                                                                                          |

`null` for array fields despite schema declaring arrays,  
consistently across all retries

returned schema-valid arrays

- 6 `_JsonFormatter` 's field allow-list dropped 6 operational  
fields from log output

Log *content* assertions in tests  
didn't check the allow-list  
exhaustively

**2026-07-16** — `getMailBody` **fetch-path validation**, ticket 239908 (5 emails, 1 PNG attachment):  
`HttpEmailGateway` fetched and parsed all 5 emails correctly across 3 CLI attempts, zero adapter errors,  
prompt built at ~1612 tokens. Two **unrelated** blockers prevented a full write on this run (not  
regressions):

- No `tesseract-ocr` on local dev PATH → the PNG attachment degrades to **FAILED** extraction  
(documented, expected without the system package).
- RunPod worker stuck in `initializing` for 20+ minutes across all 3 attempts (full 300s client-side  
timeouts, not the fast casing-bug 404) — infra-side, parked pending RunPod console access.  
Confirmed harmless to correctness: the CLI exits before any DB write happens, so no bad data  
landed.

---

## 10. Quality gates (verified just now)

```
uv run pytest -q          → 184 passed, 8 deselected (integration, needs Docker)
uv run mypy --strict src → Success: no issues found in 34 source files
uv run ruff check src     → All checks passed
```

No CI pipeline yet — all of the above is run locally/on-demand only.

---

## 11. Working-tree state (uncommitted right now)

- `CLAUDE.md`, `src/summarizer/adapters/email/http_email_gateway.py`,  
`src/summarizer/config/settings.py`, `src/summarizer/domain/ports.py`,  
`tests/unit/adapters/email/test_http_email_gateway.py` — the `getMailBody` contract  
migration described in §8, already passing all tests/mypy/ruff, validated against real staging data  
for the fetch path. Ready to commit once the RYW-signal question is either confirmed or explicitly  
accepted as an open risk.
- `logs-vLLM v2.14.0.txt`, `new_context.txt` — untracked diagnostic artifacts (a stale healthy-run  
log, and the raw `getMailBody` sample payload used to build the parser). Reference material, not  
part of the package; not committed.

---

## 12. Open questions / next steps

1. **Confirm the RYW "not yet available" signal shape** against a real not-yet-available ticket on stage  
— only unresolved piece of the `getMailBody` migration.
2. **Get the RunPod worker un-stuck** (console access needed) to complete the 239908 end-to-end  
write.
3. **Install** `tesseract-ocr` in whatever image ships the worker (Docker/AMI) before OCR extraction  
works outside this dev machine.



4. **Run the 8 MySQL integration tests** somewhere with Docker (CI, or a dev box that has it) before trusting the CAS concurrency/locking behavior under real contention.
5. **Wire up SQS** (owned by the team, not blocking further pipeline work) — the CLI's orchestrator call is already the exact shape the consumer will need.
6. Product decision still open: if a **non-triggering** email is permanently unfetchable (not the transient